AdaptiveOps — Automated E-Commerce Deployment Platform

Requirements Gathering

1. Stakeholder analysis

Stakeholder	Need / interest	How requirements address it
Demo customer / end user	Accessible storefront; browsing, cart and mock checkout without real charges	Verify frontend/cart/checkout; test product search separately at its service API
Developer / release operator	Repeatable build, tests, rollout and controlled recovery	Jenkins stages, deployment verification and rollback
Operations / monitoring engineer	Health metrics, dashboards, capacity response and incident evidence	Prometheus, Grafana, Kubernetes HPA
Infrastructure operator	Repeatable local/cloud environment setup with safe permissions	Ansible, Terraform, access and cost checks

2. User stories

ID / actor	User story	Acceptance evidence
US-01 Customer	As a visitor, I can access the deployed shop and see products so I know the frontend is serving traffic.	Accessible URL and verified product page
US-02 Tester	As a tester, I can call SearchProducts and check returned products against the sample catalog.	gRPC request/response evidence; explicitly documented API-level search scope
US-03 Customer	As a visitor, I can add an item to a cart and simulate checkout without real payment.	Cart and mocked payment response
US-04 Developer	As a contributor, I can submit a change that passes through automated checks and a container build.	Commit/PR, Jenkins console logs
US-05 Release operator	As an operator, I can deploy a known version and see whether it is healthy.	Image tag, rollout and smoke-test logs
US-06 SRE	As an operator, I can observe workload health and demand on dashboards.	Prometheus target status; Grafana panels
US-07 SRE	As an operator, I can generate load and observe automatic replica scaling.	Load logs; HPA replica progression
US-08 SRE	As an operator, I can see a faulty deployment detected and automatically reverted.	Failing checks, rollback logs, successful recovery
US-09 Infra operator	As an operator, I can reproduce host or optional AWS setup using versioned code.	Playbook output / Terraform plan and safe resources

Use-case descriptions (supporting Section 2)

UC-01 - Customer demonstration: A visitor accesses the deployed storefront, views product details, adds an item to the cart, and completes simulated checkout. Expected result: the page, cart and mock-payment responses are verified.

UC-02 - Automated deployment: A contributor submits an approved code/configuration change. Jenkins runs checks, builds a traceable image, deploys it to Kubernetes and executes smoke tests. Expected result: the new release is accepted only after validation.

UC-03 - Reliability demonstration: A tester generates load to verify HPA behavior, then introduces a controlled faulty release. The configured pipeline detects failure, restores a known-good revision and repeats smoke tests. Expected result: recorded scaling, rollback and recovery evidence.

3. Functional requirements

ID	Proposed requirement
FR-01	Maintain all team code, configs and documents in the assigned Git repository with source attribution.
FR-02	Preserve reproducible deployment of an explicitly pinned Google Online Boutique revision.
FR-03	Verify storefront, SearchProducts API, mock payment and Redis-backed cart behavior; document the scope and storage limitations.
FR-04	Build or adapt container images with documented Docker procedures.
FR-05	Automatically execute Jenkins CI tests and container-image build on a controlled trigger.
FR-06	Publish versioned artifacts/images with enough metadata to identify the released revision.
FR-07	Deploy the app and required dependencies to Kubernetes with version-controlled manifests/chart/config.
FR-08	Expose the application through a configured Nginx reverse proxy / routing entry.
FR-09	Collect selected application or infrastructure metrics in Prometheus.
FR-10	Provide Grafana dashboards sourced from Prometheus to show measured trends.
FR-11	Configure Kubernetes HPA with valid metric sources and scale-out/down controls.
FR-12	Generate repeatable load and record actual scaling behavior and client access.
FR-13	Configure hosts/services using reproducible Ansible playbooks.
FR-14	Provide and validate Terraform scripts for feasible AWS resources, with cost checks and cleanup; apply only when available and authorized.
FR-15	Detect a defined rollout or functional failure and automatically restore the retained known-good image/config.
FR-16	Verify recovery with functional smoke tests and preserve failure/rollback logs. Record any configuration or data limits.
FR-17	Record setup, environment prerequisites, team contributions, tests and known limitations.

4. Non-functional requirements (targets to validate)

ID / category	Proposed expectation	How to assess
NFR-01 Reproducibility	Same documented configuration can be deployed twice from clean target state.	Two installation/deployment traces
NFR-02 Observability	At least one working Grafana dashboard with timestamped live metrics and visible target health.	Prometheus targets; Grafana screenshot
NFR-03 Reliability	An unhealthy release triggers rollback and functional recovery checks; a static health endpoint alone is insufficient.	Failure injection and health check
NFR-04 Scalability	At least one eligible workload scales from 1 to >=2 replicas under a valid metric, then returns toward minimum after load ends.	HPA/metrics/load timeline
NFR-05 Security	No secrets in repo; use env/Secret/config templates and least-privilege credentials.	Repo inspection/config review
NFR-06 Maintainability	Config/scripts are versioned, commented where needed, and support per-environment changes.	Repository review; rerun procedure
NFR-07 Cost control	Cloud changes have a price/permission check and documented destroy/cleanup procedure.	Cost/plan logs; cleanup evidence
NFR-08 Portability	The main demonstration works in local Ubuntu VM, independent of AWS session timing.	Local run demo
NFR-09 Usability	Another teammate can follow README/runbook and reproduce a known deployment.	Peer execution checklist
NFR-10 Performance	Record throughput, p95 latency and error rate for defined baseline/load runs; agree thresholds after feasibility.	Fixed workload/duration, VM allocation and timestamped results